

What is an interface?

In everyday language, the word “interface” means a boundary between two things, which may interact with each other across that boundary.

In computer programming, the word **interface** has at least two specific technical meanings:

- A **user interface** (UI) is a boundary between a program and the human who uses it.
- An **application programming interface** (API) is a boundary between one piece of code and another piece of code.

An interface is more than just a boundary between X and Y — it defines the possible “ways” for X to interact with Y:

- A text-based user interface has a set of commands which the user may enter via the keyboard; these are the *only* commands the program accepts from the user.
- A graphical user interface (GUI) has buttons, menus and other controls which the user may interact with; these are the *only* ways the user may interact with the program.
- An API supports a set of methods or requests, which may be invoked by other code; these are the *only* methods or requests supported.

In complex software systems, we try to separate the **main program** itself (including the UI) from the **model**, and do the hard work in the model in order to keep the UI code simple. Therefore, the vast majority of code doesn’t directly interact with the user at all. However, all code interacts with some other code; the program interacts with the model, and model classes interact with other model classes.

From now on, we will only consider interfaces between different parts of a Java program; not user interfaces, nor interfaces between two different programs.

What is an API?

According to the principle of **encapsulation**, classes interact with each other by calling public methods — not by accessing fields (which are private). A class's public methods are its "API"; they define how the class may be used by other classes.

When we're thinking about the class's API, we only think about what matters to the "outside world" — that is, other classes which may call those methods:

- What public methods does the class have?
- What is the purpose of each method? Why should another class use it? What problem does it solve?
- What parameters does the method take?
- What does the method return?
- What exceptions can the method throw, in what circumstances?

All of these things define how the method can be called, and what the result will be — the caller needs to know them.

However, as long as the method does what it's supposed to do, the caller doesn't need to know *how* the method actually works. The body of the method — that is, the actual code executed when the method is called — is an implementation detail, hidden on the "other side" of the interface.

A car is a good analogy. When you're driving the car, you need to know that there is a steering wheel, an accelerator pedal and a brake pedal, and you need to know what will be the effect of turning the steering wheel or pressing the pedals down.

Of course, there are physical mechanisms inside the car to achieve those effects; but they are hidden, and irrelevant to the driver.

For example, consider the following simple class, which models an attendance register:

```
1: import java.util.*;
2:
3: public class AttendanceRegister {
4:     private final List<String> names = new ArrayList<>();
5:
6:     public void addName(String name) throws AlreadyPresentException {
7:         if(names.contains(name)) {
8:             throw new AlreadyPresentException(name);
9:         }
10:        names.add(name);
11:    }
12:
13:    public List<String> getNames() {
14:        return Collections.unmodifiableList(names);
15:    }
16: }
```

This class's API has two methods:

- The `addName` method adds a name to the register. It takes a name as a parameter; an `AlreadyPresentException` is thrown if the name has already been added.⁽¹⁾
- The `getNames` method returns an unmodifiable list of names added to the register, in the order they were added.

Note that this is exactly the same information which would appear in a specification for this class. There is no need to specify how the methods should work, only how they should be called, and what their result should be.

Message passing

What does it *mean* to call an object's method? This is a philosophical question, but it's an important one for understanding object-oriented programming.

In the imperative programming paradigm, we write **instructions** — each statement tells the program what to do and how to do it. When you call a procedure (or “function”), there is one and only one thing which can happen: the procedure's instructions are executed.

In contrast, consider what happens when you open a web page in your browser:

1. The browser sends a request to the server.
2. ???
3. The server sends a response back to the browser.

The second step is not completely unknown — but there are many different things the server might do, and many different ways the server might do them. We don't necessarily know what software is running on the server, or how that software is written. However, the browser *does* know how to send the request and how to receive the response. It is easier to think of the web request as a **message** from one piece of code to another.

There are a few philosophical differences between these two scenarios:

- You know what instructions the procedure will execute — but you don't know what instructions the web server will execute.
- You **tell** the procedure to execute, but you **ask** for a response from the web server. You have no power and no responsibility over the server.
- Calling the procedure means the same thing as executing its instructions — the procedure **is** its instructions. On the other hand, the request is not the same thing as the instructions for responding to it.

In the object-oriented programming paradigm, calling a method is somewhere between these two extremes. On one hand, when we call a method we may know what instructions will be executed. On the other hand, we don't *need* to know those instructions — it's the object's responsibility to respond to the method properly.

Methods are not instructions

In fact, sometimes when you call a method you *don't* know what instructions will be executed. Consider the following code:

```
"Hello".toString()
    "Hello" (String)
import java.util.*;
List<Integer> numbers = new ArrayList<>();
numbers.add(1);
numbers.add(2);
numbers.add(3);
numbers.toString()
    "[1, 2, 3]" (String)
AttendanceRegister register = new AttendanceRegister();
register.toString()
    "AttendanceRegister@1e2484b" (String)
```

This code needs some explaining.

Every object has a `toString` method, even if you don't declare this method in your own class. But different objects do different things when you call their `toString` method:

- The value `"Hello"` is already a `String`, so its `toString` method simply returns the string itself.
- The `ArrayList` class's `toString` method converts the list to a string, with the list elements separated by commas inside square brackets.
- The `AttendanceRegister` class doesn't have its own implementation of `toString`, so it uses the default implementation, with the result shown above.

This means different classes can have different **implementations** of the same method. The `toString` method is not the same as the instructions for how to convert an object to a string — because there is one method, but many different sets of instructions.

This makes sense if we think about message passing: we send a `toString` message to the object, and the object sends a string back. Every object accepts the `toString` message, but responds in its own way: when we call the `toString` method, we are not **telling** the program how to convert the object to a string — we are **asking** the object to convert itself to a string.

Thinking of method calls as messages, a class's interface defines what kinds of messages an object will accept, and what kinds of responses the caller should expect.